

Exact Unlearning of Finetuning Data via Model Merging at Scale

Kevin Kuo*, Amrith Setlur[†], Kartik Srinivas[†], Aditi Raghunathan*, Virginia Smith[†]
Carnegie Mellon University

Abstract

Approximate unlearning has gained popularity as an approach to efficiently update an LLM so that it behaves (roughly) as if it was not trained on a subset of data to begin with. However, existing methods are brittle in practice and can easily be attacked to reveal supposedly unlearned information. To alleviate issues with approximate unlearning, we instead propose SIFT-Masks (SIgn-Fixed Tuning-Masks), an *exact unlearning* method based on model merging. SIFT-Masks addresses two key limitations of standard model merging: (1) merging a large number of tasks can severely harm utility; and (2) methods that boost utility by sharing extra information across tasks make exact unlearning prohibitively expensive. SIFT-Masks solves these issues by (1) applying local masks to recover task-specific performance; and (2) constraining finetuning to align with a global sign vector as a lightweight approach to determine masks independently before merging. Across four settings where we merge up to 500 models, SIFT-Masks improves accuracy by 5-80% over naïve merging and uses up to 250 $\times$ less compute for exact unlearning compared to other merging baselines.

1 Introduction

Modern machine learning applications often require finetuning a pretrained model on a collection of data. However, once a model has been finetuned, it may be necessary to *unlearn* a subset of data and produce a model identical to one trained as if the data were never present. This is because finetuning data can introduce risks such as harmful knowledge or private information (Carlini et al., 2023; More et al., 2024; Su et al., 2024; Ahmadian et al., 2024). Moreover, data privacy regulations such as GDPR and CCPA state that consumers have a “right to be forgotten” (Protection, 2018). For ML, this not only requires that data controllers remove data in accordance with deletion requests, but also retrain any models trained on such data. To address these concerns, there has been significant interest in methods for *machine unlearning* that can efficiently remove the influence of data from a model (Cao & Yang, 2015; Ginart et al., 2019; Bourtole et al., 2021; Tarun et al., 2023). However, existing methods face key limitations: approximate unlearning methods lack guarantees—leading to exposure of supposedly unlearned information (Hu et al., 2025; Lucki et al., 2024; Deeb & Roger, 2024), while exact unlearning methods have prohibitively expensive relearning costs.

In this work, we explore using *model merging* for both exact and efficient unlearning. Given a dataset split over several tasks (e.g., a large set of clients whose data we may wish to unlearn), we first finetune (FT) a pretrained model separately on each task to obtain a set of

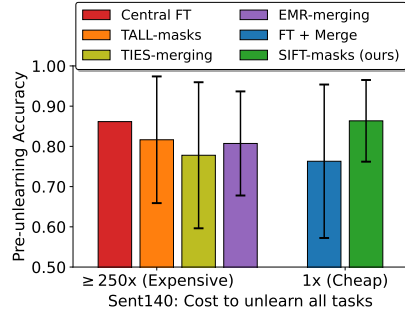


Figure 1: SIFT-Masks is a merging-and-localization method that can match central training accuracy while being as efficient for unlearning as naïve merging.

*Computer Science Department. Direct correspondence to Kevin Kuo (kkuo2@andrew.cmu.edu).

[†]Machine Learning Department

local models. While it is easy to unlearn a task by discarding its local model, this framework is limited by both high storage costs and lack of collaboration across tasks. Therefore, we *merge* (average) the local models’ weights to produce a single *merged* model and then discard the local models. To unlearn a particular task, we can simply retrain the local model for that task and *unmerge* (subtract) it from the merged model. This results in a model where the task has been unlearned exactly, *i.e.*, it matches the merged model from the remaining tasks.

In Figure 1, we show that naïve averaging of this form (FT + Merge) is indeed a cheap way to enable exact unlearning—resulting in unlearning costs that are $250\times$ more efficient than performing exact unlearning by retraining a model on all the revised data (Central FT). However, the accuracy of FT + Merge is poor. It is thus natural to consider whether recent merging approaches could boost accuracy while keeping unlearning costs low. Unfortunately, sophisticated merging methods which are more effective at scale like EMR-merging (Huang et al., 2024) or TALL-masks (Wang et al., 2024) rely on *localization*—a method to recover performance by applying task-specific masks to the merged model. As shown in Figure 1, while these methods improve accuracy, they come at a high cost: current localization methods must relearn all local models to reconstruct local masks after a task is removed, rendering them as expensive (if not more so) than the naïve Central FT baseline.

To address these challenges, we propose SIFT-Masks (**S**ign-**F**ixed **T**uning **M**asks), a lightweight model merging method that is uniquely suited for large-scale unlearning. SIFT-Masks initializes a global random sign vector and constrains the entries within a task vector to agree with this vector while setting the others to zero. Notably, the global sign vector is chosen independently of the tasks and their data (*i.e.*, before any training begins). This design choice substantially improves the cost of unlearning, matching that of the efficient naïve averaging baseline. In summary, our contributions are as follows:

1. We merge up to 500 models, which is more than an order of magnitude more tasks than prior work (Ilharco et al., 2023; Wang et al., 2024). Our setting reflects the realistic scenario in unlearning where models are finetuned over a large set of tasks (e.g., a pool of clients or collection of documents), and tasks contribute subtly differing data for a common learning task rather than the tasks being largely disjoint.
2. We identify a key deficiency in using localization-based merging for unlearning: current localization methods boost accuracy when merging by sharing extra information across tasks, but this information sharing makes exact unlearning computationally infeasible.
3. We propose SIFT, a finetuning method which makes model merging and localization computationally feasible for exact unlearning. Unlike existing methods where masks require global information, SIFT-Masks obtains masks by using only a random sign vector and local data. This allows us to maintain accuracy while enabling efficient exact unlearning at scale: As we show through extensive experiments on unlearning tasks, SIFT-Masks improves accuracy by 5-80% over naïve merging and uses up to $250\times$ less compute for unlearning compared to other merging baselines.

2 Related Work

Machine Unlearning. Unlearning benchmarks typically consider unlearning over a large number of tasks such as users who wish to opt out of data sharing or data sources which are found to contain harmful information (Li et al., 2024a; Jin et al., 2024; Maini et al., 2024). Therefore, it is important to design methods that can efficiently update a model in response to multiple deletion requests. Standard approaches for *exact* unlearning tend to have high computational costs from retraining over a large retain set or high storage costs from maintaining and ensembling models trained on disjoint shards of data (Bourtoule et al., 2021; Yan et al., 2022; Chen et al., 2022; Li et al., 2024b; Chowdhury et al., 2024). On the other hand, *approximate* unlearning methods do not provably remove the influence of data points from the supposedly unlearned model and are only evaluated via empirical tests (Eldan & Russinovich, 2024; Liu et al., 2024). Consequently, many prior works show that approximate unlearning approaches are brittle and can be easily attacked (Marchant et al., 2022; Bertran et al., 2024; Hu et al., 2024a; 2025; Ginart et al., 2019; Bourtoule et al.,

Method	Description	Unlearn Cost	Storage
Central FT	Train a single model on all pooled data	T	M
<i>Flexible sharding</i>			
SISA (Bourtoule et al., 2021)	Ensemble models trained on random data shards	T/S	MS
APA (Hu et al., 2024b)	Merge models trained on data-driven shards	T	MS
<i>Fixed task-level shards</i>			
FT + Merge (Ilharco et al., 2023)	Merge models trained on task-level shards	1	M
TIES-merging (Yadav et al., 2024a)	Reduces sign conflicts from FT + Merge	T	M
TALL-masks (Wang et al., 2024) EMR-merging (Huang et al., 2024)	Construct masks using merged and local models	T	$M(1 + \frac{T}{32})$
SIFT-Masks (this work)	Construct masks using only local models	1	$M(1 + \frac{T}{32})$

Table 1: Comparison of (1) exact unlearning methods based on merging/ensembling; and (2) model merging methods which can be applied to exact unlearning. T is the number of retained tasks, S is the number of shards (disjoint partition of tasks), and M is the number of model parameters. “Unlearn Cost” is the worst-case cost of unlearning a single task in terms of the number of tasks we must finetune over. In this work, we focus on methods that assume fixed task-level shards. Masking methods store a mask for each task which costs $1/32$ the size of a full model.

2021; Tarun et al., 2023). Unlearning is also a natural problem in distributed or federated settings where users benefit from sharing their data or model parameters; methods tailored to these settings can similarly be categorized as exact (Qiu et al., 2023; Xiong et al., 2023; Xia et al., 2024) or approximate (Wu et al., 2022; Halimi et al., 2022).

Model Merging. Model merging is a promising approach to enable exact unlearning. Early work in model merging averages the parameters of multiple models trained with different hyperparameters on the same data to improve generalization (Wortsman et al., 2022). Concurrent works extend this method to multi-task learning by training models on diverse vision tasks and averaging their weights (Matena & Raffel, 2022; Dimitriadis et al., 2023; Ilharco et al., 2023). Since then, many methods have been proposed to improve the quality of naive merging (FT + Merge), such as linearized finetuning (Ortiz-Jimenez et al., 2024) and sparsifying task vectors (Marczak et al., 2024; Yu et al., 2024; He et al., 2024; Davari & Belilovsky, 2025), or selectively merging subsets of weights (Ainsworth et al., 2022; Stoica et al., 2023; Ye et al., 2023; Xu et al., 2024). However, adding complexity to FT + Merge can make it unsuitable for exact unlearning. For example, TIES-merging (Yadav et al., 2024a) is similar in spirit to our work; it seeks to minimize sign conflicts in FT + Merge. A crucial step of TIES-merging is *electing* a global sign based on all models before merging occurs, but this dependence between models prior to merging makes exact unlearning non-trivial.

Scaling merging to a large number of models is an open question; current works focus more on the benefits of scaling the model size while merging relatively few tasks (Yadav et al., 2024b). While prior works have proposed using merging for unlearning, they mostly consider an approximate unlearning setting where the goal is to remove pretrained knowledge from the model. Specifically, Ilharco et al. (2023); Kim et al. (2024) apply a negated task vector which roughly approximates gradient ascent on the pretrained model, while Kadhe et al. (2024) considers merging multiple approximately unlearned models. In contrast, our work focuses on exact unlearning of additional knowledge acquired during finetuning.

Model Localization. Localization applies task-specific masks which can recover much of the performance lost during model merging (Wang et al., 2024; Huang et al., 2024). In general, this area of work uses extra storage to preserve task-specific models. Other similar works improve upon naively storing all of the local models by storing multiple models (Zhang et al., 2024; Hu et al., 2024b; Lu et al., 2024). However, all of these works are limited in scale: Wang et al. (2024) and Huang et al. (2024) merge up to 30 models and all the other works we are aware of only merge up to 8 models.

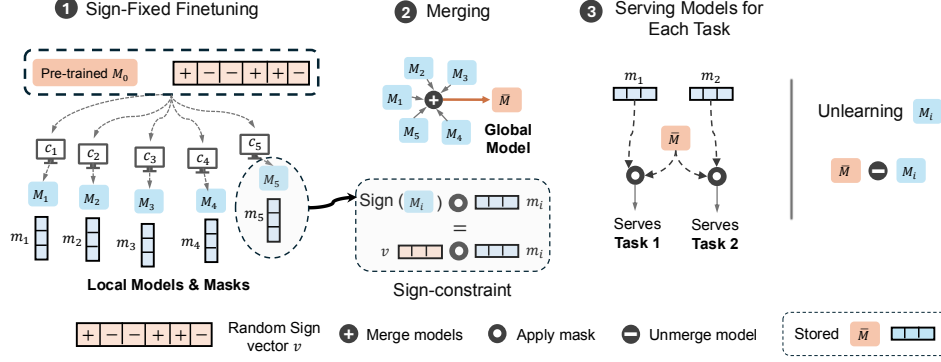


Figure 2: SIFT-Masks starts with (1) **Sign-Fixed (Fine)-Tuning**, which initializes a random sign vector v and constrains finetuning to match this sign vector or otherwise be sparse, producing a sparse local model M_i with mask m_i . We then (2) merge these local models into a global model and only keep the masks m_i . When (3) serving task c_i , we apply m_i to the merged model. Finally, to unlearn task c_i , we simply unmerge M_i from $\bar{M}$ and discard m_i .

We critically note that subtle design choices made by a few methods in Table 1 can result in unlearning costs similar to naïve central training. APA (Hu et al., 2024b) proposes *data-driven clustering*, while TALL-masks (Wang et al., 2024) and EMR-merging (Huang et al., 2024) propose *masks which depend on the merged model*. Due to the extra information these methods share across tasks, subtracting model weights is insufficient to remove the influence of a task. For example, removing a client from a cluster may change the clusters output by the clustering algorithm, making it necessary to re-run the entire learning algorithm.

3 SIFT-Masks: Sign-Fixed Tuning and Masking

As discussed, in this work we focus on *exact* unlearning methods which have the benefit of provable unlearning by design. Given a dataset composed of a collection of tasks $c_1, \dots, c_T$, a learning algorithm $\mathcal{A}$, and model $M = \mathcal{A}(\cup_{i \in [T]} c_i)$, the goal of exactly unlearning task c_u is to construct a new model $M_u = \mathcal{A}(\cup_{i \in [T] \setminus \{u\}} c_i)$ that matches the output from running $\mathcal{A}$ as if c_u never existed.

Model merging provides both efficient and exact unlearning by design. Merging is a framework which finetunes a pretrained model M_0 separately on several tasks, constructs residual *task vectors* $\tau_c = M_0 - M_c$, and then combines these to produce a multi-task vector $\bar{\tau} = \sum_{c \in [T]} \tau_c$ and a merged model $\bar{M} = M_0 + \bar{\tau}$ (Ilharco et al., 2023). Under this framework, we can unlearn task c_u by simply subtracting its task vector which exactly yields $\bar{\tau} - \tau_u = \sum_{c \in [T] \setminus \{u\}} \tau_c$, the merged model as if c_u were never present. To avoid storing all the task vectors, we only store $\bar{\tau}$ and retrain τ_u when a deletion request is made. This retraining must be deterministic (e.g. initialization and data sampling) in order to reproduce the same weights of τ_u .

Localization recovers utility lost from merging. Prior work has shown that a merged model $\bar{M}$ tends to perform worse than the task-level models. In Figure 3, we show that as the number of models increases, performance can degrade even further. **Localization** is a promising approach to recover this lost performance; localization-based methods learn an additional mask m_t for each task which approximates the local model weights once applied to the multi-task vector $M_t \approx M_0 + m_t \odot \bar{\tau}$. We compare to two baselines: TALL-masks, which merges by averaging and localizes by using a similarity threshold hyperparameter λ_t : $m_t = \mathbb{1}\{|\tau_t| \geq |\bar{\tau} - \tau_t| \cdot \lambda_t\}$ (Wang et al., 2024); and EMR-merging, which merges by taking a maximum over all weights which align with the average sign, localizes with sign agreement $m_t = \mathbb{1}\{\tau_t \odot \bar{\tau} > 0\}$, and then rescales $m \odot \bar{\tau}$ to match the ℓ_1 norm of τ_t (Huang et al., 2024).

Existing localization methods are unsuitable for exact unlearning. A critical limitation of localization is that masks depend on both the local and merged models. To satisfy exact unlearning, the local masks cannot directly be reused after an unlearning request is made

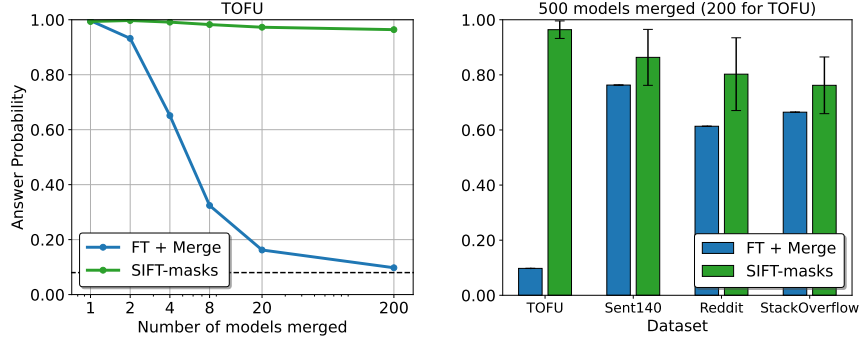


Figure 3: Left: Merging ($x > 1$) degrades performance (answer probability) compared to applying local models ($x = 1$); this issue becomes more severe as the number of models increases, potentially reducing performance to zeroshot accuracy. Right: Our method SIFT-Masks recovers performance (probability for TOFU; accuracy otherwise) from merging and suffers much less from scale.

and instead must be reconstructed using the new merged model. Furthermore, in order to reconstruct the masks, each local model has to be retrained, since we do not store the local models. Therefore, naively applying these methods for exact unlearning requires finetuning over the entire dataset after each deletion request, which is computationally infeasible.

SIFT-Masks resolves the tension between localization and exact unlearning. Given the above challenges, our goal is to construct local masks in a manner which depends **only** on the local model and **not** the merged model. To do this, we propose SIFT (Sign-Fixed Tuning), which is shown in Figure 2. The first step of SIFT is to initialize a uniformly random sign vector v that is shared across all tasks. During local finetuning, we constrain the entries of each task vector τ_c such that $\tau_c \odot v$ is greater than or equal to 0. We project τ_c to this constraint set after each gradient step by clipping all entries of τ_c to 0 where $\tau_c \odot v < 0$. After finetuning, all task vector weights will either be 0 or share the same sign as v , which produces the local mask $m_c = \mathbb{1}\{\tau_c \odot v > 0\}$. Our complete method SIFT-Masks (pseudocode in Appendix A) applies these masks to the merged task vector.

4 Results

In this section we empirically explore the performance of SIFT-Masks. We first consider the accuracy of SIFT-Masks for merging at scale, comparing SIFT to regular finetuning (FT) in Sec. 4.1. We then analyze unlearning costs, measuring the cost of unlearning a single task across methods in Sec. 4.2, and unlearning multiple tasks in Sec. 4.3. Finally, we evaluate simple alternatives to improve the naïve baselines (Central FT and FT + Merge) in Sec. 4.4. We finetune models ranging from 700M to 1.5B parameters on four common text datasets from the unlearning/federated learning literature, where unlearning requests from tasks or clients naturally occur: TOFU (Maini et al., 2024), Sent140 (Go et al., 2009), Reddit (Caldas et al., 2018), and StackOverflow (Huggingface, 2023). We provide a few details in Table 2 with full dataset & model details in Appendix B.

Dataset	Tasks	Labels	Label Type	Total Examples
TOFU	200	50256	tokens	4000
Sent140	500	2	sentiment	40504
Reddit	500	20	subreddits	67484
StackOverflow	500	10	tags	97037

Table 2: Overview of our datasets. Each dataset is naturally partitioned over a large number of tasks.

4.1 Merging and localization accuracy

First, we show that across multiple datasets and varying numbers of merged models, SIFT-Masks has significant benefits compared to FT + Merge. In Figure 3 (left), we plot the

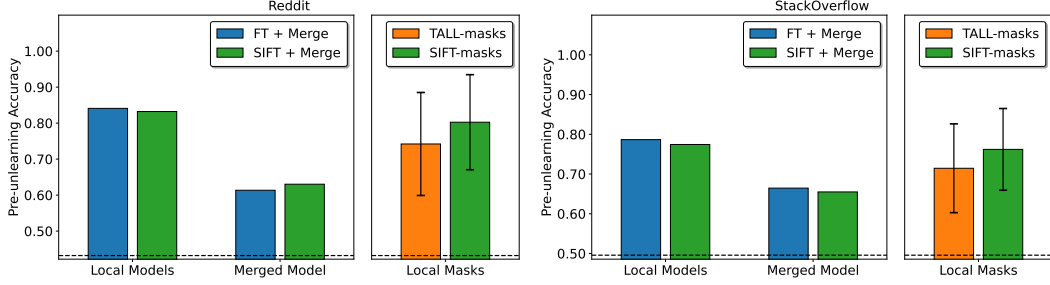


Figure 4: SIFT (Sign-Fixed Tuning) produces sparse local models (and corresponding masks) which have similar utility as regular FT (finetuning) when applied individually and after merging. However, the main benefit of SIFT is that the sparse masks can be applied to the merged model to obtain strong task-specific models. Despite learning these masks independently from the merged model (which is useful for unlearning efficiency), SIFT-Masks is competitive with existing localization approaches which optimize the mask to minimize distance between the merged and local models.

performance (answer probability) of SIFT-Masks and FT + Merge on TOFU as we vary the number of merged models from 1 to 200. At 200 merged models, FT + Merge degrades to zero-shot probability, while SIFT-Masks remains at 99% probability. On the right plot, we show the performance of these two methods at the maximum number of tasks (200 for TOFU, 500 for others). Across the other 3 datasets, SIFT-Masks recovers 5-20% accuracy.

Next, we compare SIFT + Merge to regular finetuning (FT + Merge) at a more detailed level. We show that the sign constraint of SIFT has little impact on finetuning quality in terms of both the local and merged models’ performance. In Figure 4, the performance of SIFT is similar to that of FT (Local Models). Next, although SIFT + Merge eliminates sign conflicts during merging, the large number of tasks we merge (500) still causes significant interference, resulting in similar performance as regular FT + Merge. The benefit of SIFT is only clear once the local masks are applied. Despite constructing masks using only local data, SIFT-Masks outperforms TALL-masks, a baseline which optimizes the mask to minimize distance between the merged and local models.

In Figure 5, we plot 100 Sent140 tasks as individual points and compare three different methods: regular finetuning and merging (FT + Merge), sign-fixed tuning and merging (SIFT + Merge), and the complete SIFT-Masks method (SIFT + Merge + Mask). On the x-axis, we measure the distance from the merged (and masked) model to the local model it is trying to approximate. With SIFT + Merge, we eliminate sign conflicts and obtain a merged model which is relatively closer to the local models compared to FT + Merge. However, this does not significantly improve accuracy compared to FT + Merge. This is because the merged model still contains non-zero weights in entries where the local weight is zero and the global sign points in a direction which is harmful for that task. SIFT-Masks adds task-specific masks which removes these weights and is key to improving accuracy.

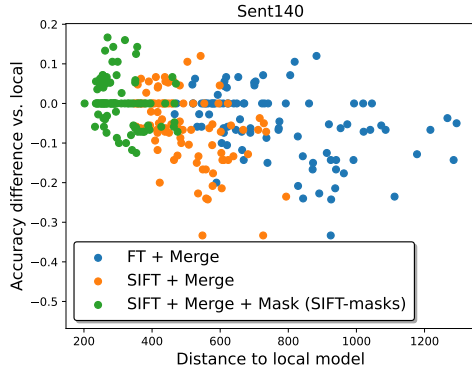


Figure 5: SIFT reduces the distance between the merged and local models, but this does not directly result in improved accuracy due to interference from large-scale merging. Instead, accuracy only improves after applying local masks.

4.2 Costs of unlearning

In this section, we compare the unlearning cost of each method. In terms of the number of finetuning steps required for exact unlearning, SIFT-Masks is more efficient than all other methods: Central FT, FT + Merge, TIES-merging (Yadav et al., 2024a), TALL-masks (Wang

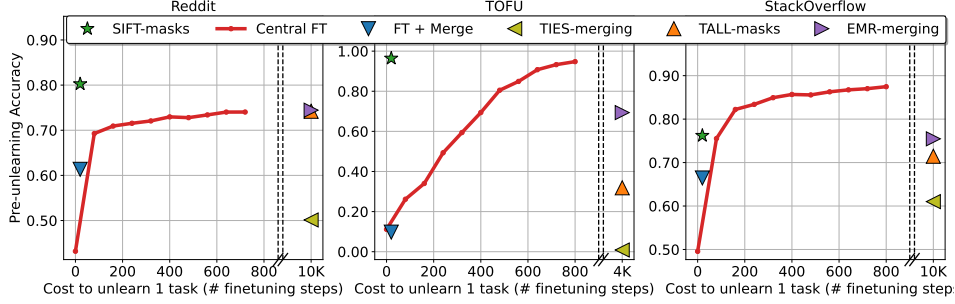


Figure 6: We compare SIFT-Masks to Central FT, Merge + FT, and TALL-masks. Depending on the setting, SIFT-Masks can outperform Central FT due to task heterogeneity (e.g. conflicting examples or distinct distributions). Across all settings, SIFT-Masks improves on the tradeoff of efficiency and accuracy compared to varying the number of finetuning steps used for Central FT.

et al., 2024), and EMR-merging (Huang et al., 2024). We finetune for up to 800 steps on Central FT and fix 20 steps for all other approaches. For T tasks, this is a total of $20T$ finetuning steps (4,000 for TOFU, 10,000 otherwise). Although this initial cost is high, it allows FT + Merge and SIFT-Masks to efficiently unlearn a task by retraining its model for 20 steps and unmerging it. However, as previously mentioned, this same technique does not result in exact unlearning for the other methods.

In Figure 6, the setting can affect how quickly Central FT converges and its accuracy relative to SIFT-Masks. We conjecture that this is due to *data heterogeneity*: the degree to which the data for one task is helpful for another task. On Reddit, tasks have potentially *conflicting* data which a single model cannot handle; for example, similar generic comments can be posted across different subreddits. On TOFU, each task (author) contains highly *distinct* data; training on one task has little to no affect on the other tasks. Since TOFU measures held-in training performance, Central FT is guaranteed to reach 100%, but converges slowly due to the structure of the data. Finally, data across StackOverflow tasks is *similar*; labels in StackOverflow (e.g. programming languages) depend less on the task context (user) and more on global features (e.g. language keywords) of the comment. As a result, SIFT-Masks improves efficiency on StackOverflow, but cannot reach the performance of centralized training. Overall, these results show that *merging is most helpful when data is heterogeneous* i.e. performing well on a given task requires knowledge of the context or having finetuned on its training data. Additionally, we show that existing localization methods are an extremely poor choice for exact unlearning. Due to the nature of these localization methods, unlearning a single task requires re-running the entire merging and localization method on the remaining 499 tasks, resulting in $499 \times 20 = 9980$ finetuning steps and costing several times more than naive Central FT. Finally, TIES-merging (Section 2) performs poorly because it does not use localization, similar to SIFT + Merge in Figure 5. In Appendix C, we provide finer-grained experiments comparing TIES-merging to FT + Merge.

For all datasets, we found that tuning the number of steps individually for each task was generally not helpful and that 20 finetuning steps worked well as a fixed quantity across all tasks. To test the limits of our method’s efficiency, we run an additional ablation on the amount of finetuning steps. In Figure 7, we show that it is possible to use as few as 10 finetuning steps per task and achieve similar accuracy after merging and applying SIFT-Masks. We also note that the sign constraint slightly slows down training, as shown by the performance of FT + Merge versus SIFT + Merge at $x = 5$. However, as discussed, SIFT-Masks does not directly apply the merged model and instead combines it with a mask. Therefore, for the same finetuning cost SIFT-Masks always outperforms FT + Merge.

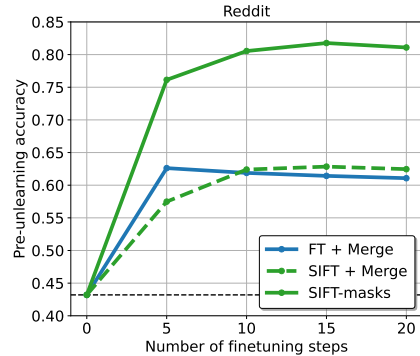


Figure 7: While we use 20 steps for local finetuning on all datasets, this amount can be carefully reduced to make merging approaches even more efficient.

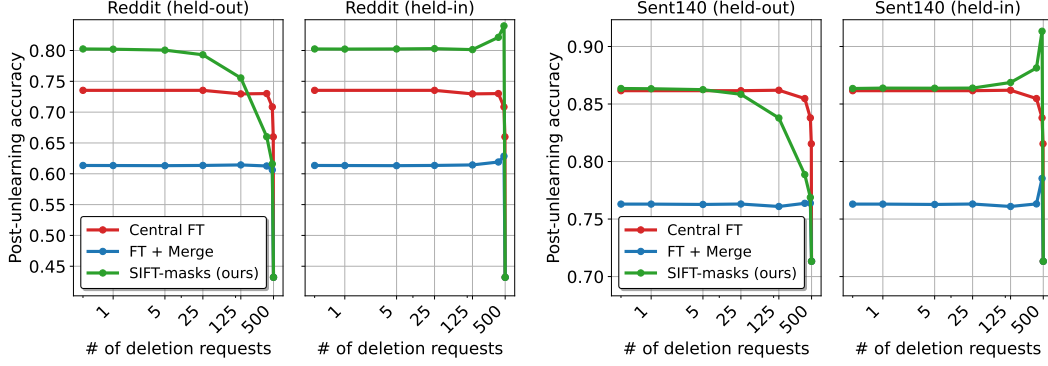


Figure 8: We evaluate two post-unlearning metrics: *held-out* performance on all clients (including unlearned clients) and *held-in* performance on the retain clients. For unlearned clients, SIFT-Masks applies the merged model without any mask. Held-out performance decreases as more tasks are unlearned, due to the poor performance of the merged model compared to SIFT-Masks. Held-in performance increases as more tasks are unlearned because of less interference in the merged model.

4.3 Handling multiple unlearning requests

Next, we evaluate the performance of various methods as multiple tasks are unlearned, ranging from 1 to all 500 tasks. In Figure 8, we show two different evaluation metrics: the first is a *held-out* evaluation that includes all 500 tasks in the dataset. To evaluate a task which has already been unlearned, SIFT-Masks applies the merged model without any mask. Since FT + Merge always worse than applying masks with SIFT-Masks, performance of SIFT-Masks **decreases** as clients are unlearned and eventually matches the performance of FT + Merge. The other metric is a *held-in* evaluation that only includes the retained tasks. Unlike the held-out setting, SIFT-Masks applies a mask to every task it is evaluated on. In this setting, performance **increases** as clients are unlearned. This is because of less interference during merging (Fig. 3), allowing SIFT-Masks to obtain better masked models. Once all clients are unlearned, performance for all methods drops to zeroshot accuracy.

When deletion requests arrive iteratively, satisfying these requests one-by-one is difficult for Central FT and leads to a large total cost. In Figure 9, we show how the cost of relearning is accumulated across up to 500 unlearning requests. We measure costs in terms of the number of tasks that require finetuning for unlearning and relearning. This metric assumes that it is necessary to train on a task’s data in order to perform well on its evaluation set. While this only true in certain settings (like on the TOFU dataset), it clearly highlights the limitations of Central FT. For example, if the unlearning system starts with 500 tasks, unlearning the first task with model merging only requires relearning that single task and unmerging its model. However, unlearning with Central FT requires learning all 499 tasks in the retain set. As more tasks are unlearned beyond this first task, this ratio becomes smaller due to the shrinking size of the retain set and reaches $250\times$ once all 500 tasks are unlearned. However, in absolute terms, this difference is very large: FT + Merge only needs to relearn 499 tasks in total, while Central FT needs to relearn $\sum_{i=1}^{499} (500 - i) \approx 125,000$ tasks. While reducing the amount of initial tasks (X initial tasks) can significantly reduce the relearning costs, this limits the number of tasks (deletion requests) the system can support. Even if we reduce the number of initial tasks to X , the cost of unlearning is still at least $(X - 1)/2$ times more expensive than merging.

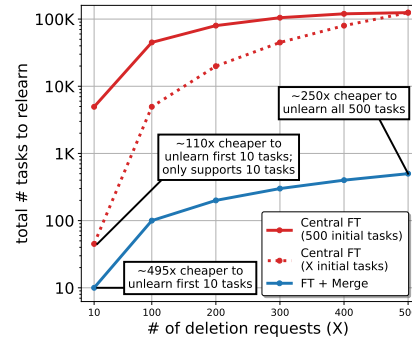


Figure 9: Training on fewer initial tasks ($X < 500$) can reduce the unlearning costs of Central FT, but limits the maximum number of deletion requests the system can support. Furthermore, the total cost of unlearning is still $\frac{X}{2}$ to X times larger than that of merging approaches. Y-axis is on a log scale.

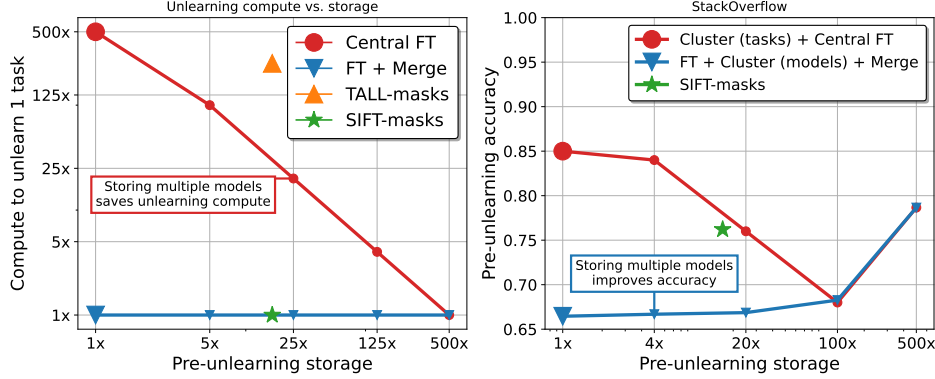


Figure 10: By paying extra storage cost to maintain multiple models, we can reduce the unlearning cost of Central FT (left) or improve the accuracy of FT + Merge (right). However, storing additional models with these two baselines is unable to match the compute or accuracy benefits of SIFT-Masks.

4.4 Compute vs. storage costs

Finally, we compare storage costs across methods and discuss the tradeoff between storage, unlearning compute, and pre-unlearning accuracy. A naive way to improve the baselines of Central FT and FT + Merge is to allow storing multiple models. Instead of a single Central FT model, we randomly cluster the tasks into equal-sized clusters and then train a model for each cluster. Similarly, instead of a single FT + Merge model, we randomly cluster the models (after FT) and merge the models in each cluster. For a given number of clusters, both of these modified methods use the same amount of storage, but vary in their compute cost and performance. The benefit of clustering in Central FT is to reduce unlearning compute, since fewer tasks have to be relearned in a given cluster. Meanwhile, the benefit of clustering in FT + Merge is to improve accuracy, since we merge less models at once and reduce merging interference. In both cases, we use random clustering because data-driven clustering cannot be trivially unlearned.

In Figure 10, we first show in the left plot that localization approaches (SIFT-Masks and TALL-masks) cost additional storage compared to single-model methods Central FT and FT + Merge. For 500 tasks, this storage cost is $500/32 \approx 16$ times that of a single model. Next, we show that clustering the tasks has limited benefits for either of the two baselines. On the left plot, learning a Central FT model on each cluster reduces unlearning costs, but the unlearning cost remains several times greater than SIFT-Masks or FT + Merge. On the right plot, merging a few number of models barely improves accuracy for FT + Merge. Additionally, clustering can lead to both worse storage and accuracy compared to SIFT-Masks ($x = 20, 100$), since clustering prevents the model from leveraging task-level context.

5 Conclusion and Future Work

In this work, we propose using model merging and localization for exact unlearning at scale. While merging is a natural framework for exact unlearning, the merged model suffers when a large number of tasks are merged, which makes additional techniques such as localization necessary. To solve existing issues with localization methods, we propose SIFT-Masks, a method that uses sign-fixed finetuning to construct local masks without depending on any cross-task information. As a result, SIFT-Masks improves the quality of merging while retaining its unlearning efficiency at scale. Overall, our work makes an important first step in identifying both the strengths and limitations of merging for exact unlearning. Our results suggest that to make model merging even more effective for unlearning, future work should focus on (1) ways to improve the quality of the merged model and (2) localization methods which are amenable to efficient unlearning. Finally, it is important to further explore the limitations of merging in general for efficient ML systems; prior work suggests that merging requires strong pretrained models, while our work shows that Central FT attains significantly higher accuracy than merging-and-localization for certain datasets.

Acknowledgements. The authors would like to thank Pratiksha Thaker and Saurabh Garg for insightful discussions on this work. This work was supported in part by the National Science Foundation grants IIS2145670 and CCF2107024, and funding from Amazon, Apple, Google, Intel, Meta, and the CyLab Security and Privacy Institute. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of any of these funding agencies. AS is thankful for the generous support of JP Morgan AI PhD Fellowship.

References

- Arash Ahmadian, Seraphina Goldfarb-Tarrant, Beyza Ermis, Marzieh Fadaee, Sara Hooker, et al. Mix data or merge models? optimizing for diverse multi-task learning. *arXiv preprint arXiv:2410.10801*, 2024.
- Samuel K Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. *arXiv preprint arXiv:2209.04836*, 2022.
- Martin Bertran, Shuai Tang, Michael Kearns, Jamie Morgenstern, Aaron Roth, and Zhiwei Steven Wu. Reconstruction attacks on machine unlearning: Simple models are vulnerable. *arXiv preprint arXiv:2405.20272*, 2024.
- Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 141–159. IEEE, 2021.
- Sebastian Caldas, Sai Meher Karthik Duddu, Peter Wu, Tian Li, Jakub Konečný, H Brendan McMahan, Virginia Smith, and Ameet Talwalkar. Leaf: A benchmark for federated settings. *arXiv preprint arXiv:1812.01097*, 2018.
- Yinzhi Cao and Junfeng Yang. Towards making systems forget with machine unlearning. In *2015 IEEE symposium on security and privacy*, pp. 463–480. IEEE, 2015.
- Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying memorization across neural language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- Chong Chen, Fei Sun, Min Zhang, and Bolin Ding. Recommendation unlearning. In *Proceedings of the ACM Web Conference 2022*, pp. 2768–2777, 2022.
- Somnath Basu Roy Chowdhury, Krzysztof Choromanski, Arijit Sehanobish, Avinava Dubey, and Snigdha Chaturvedi. Towards scalable exact machine unlearning using parameter-efficient fine-tuning. *arXiv preprint arXiv:2406.16257*, 2024.
- MohammadReza Davari and Eugene Belilovsky. Model breadcrumbs: Scaling multi-task model merging with sparse masks. In *European Conference on Computer Vision*, pp. 270–287. Springer, 2025.
- Aggyad Deeb and Fabien Roger. Do unlearning methods remove information from language model weights? *arXiv preprint arXiv:2410.08827*, 2024.
- Nikolaos Dimitriadis, Pascal Frossard, and François Fleuret. Pareto manifold learning: Tackling multiple tasks via ensembles of single-task models. In *International Conference on Machine Learning*, pp. 8015–8052. PMLR, 2023.
- Ronen Eldan and Mark Russinovich. Who’s harry potter? approximate unlearning for LLMs, 2024. URL <https://openreview.net/forum?id=PDct7vrcvT>.
- Antonio Ginart, Melody Guan, Gregory Valiant, and James Y Zou. Making ai forget you: Data deletion in machine learning. *Advances in neural information processing systems*, 32, 2019.
- Alec Go, Richa Bhayani, and Lei Huang. Twitter sentiment classification using distant supervision. *CS224N project report, Stanford*, 1(12):2009, 2009.

- Anisa Halimi, Swanand Ravindra Kadhe, Ambrish Rawat, and Nathalie Baracaldo Angel. Federated unlearning: How to efficiently erase a client in fl? In *International Conference on Machine Learning*, 2022.
- Yifei He, Yuzheng Hu, Yong Lin, Tong Zhang, and Han Zhao. Localize-and-stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.
- Hongsheng Hu, Shuo Wang, Tian Dong, and Minhui Xue. Learn what you want to unlearn: Unlearning inversion attacks against machine unlearning. *arXiv preprint arXiv:2404.03233*, 2024a.
- Shengyuan Hu, Yiwei Fu, Steven Wu, and Virginia Smith. Unlearning or obfuscating? jogging the memory of unlearned llms via benign relearning. In *International Conference on Learning Representations*, 2025.
- Zhiyu Hu, Yang Zhang, Minghao Xiao, Wenjie Wang, Fuli Feng, and Xiangnan He. Exact and efficient unlearning for large language model-based recommendation. *arXiv preprint arXiv:2404.10327*, 2024b.
- Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. Emr-merging: Tuning-free high-performance model merging. *arXiv preprint arXiv:2405.17461*, 2024.
- Huggingface. mikex86/stackoverflow-posts, June 2023. URL <https://huggingface.co/datasets/mikex86/stackoverflow-posts>.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023.
- Zhuoran Jin, Pengfei Cao, Chenhao Wang, Zhitao He, Hongbang Yuan, Jiachun Li, Yubo Chen, Kang Liu, and Jun Zhao. Rwku: Benchmarking real-world knowledge unlearning for large language models. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- Swanand Ravindra Kadhe, Farhan Ahmed, Dennis Wei, Nathalie Baracaldo, and Inkit Padhi. Split, unlearn, merge: Leveraging data attributes for more effective unlearning in llms. *arXiv preprint arXiv:2406.11780*, 2024.
- Hyoseo Kim, Dongyoon Han, and Junsuk Choe. Negmerge: Consensual weight negation for strong machine unlearning. *arXiv preprint arXiv:2410.05583*, 2024.
- Nathaniel Li, Alexander Pan, Anjali Gopal, Summer Yue, Daniel Berrios, Alice Gatti, Justin D Li, Ann-Kathrin Dombrowski, Shashwat Goel, Long Phan, et al. The wmdp benchmark: Measuring and reducing malicious use with unlearning. *arXiv preprint arXiv:2403.03218*, 2024a.
- Yuyuan Li, Chaochao Chen, Xiaolin Zheng, Junlin Liu, and Jun Wang. Making recommender systems forget: Learning and unlearning for erasable recommendation. *Knowledge-Based Systems*, 283:111124, 2024b.
- Jiancheng Liu, Parikshit Ram, Yuguang Yao, Gaowen Liu, Yang Liu, PRANAY SHARMA, Sijia Liu, et al. Model sparsity can simplify machine unlearning. *Advances in Neural Information Processing Systems*, 36, 2024.
- Zhenyi Lu, Chenghao Fan, Wei Wei, Xiaoye Qu, Danyang Chen, and Yu Cheng. Twin-merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.
- Jakub Łucki, Boyi Wei, Yangsibo Huang, Peter Henderson, Florian Tramèr, and Javier Rando. An adversarial perspective on machine unlearning for ai safety. *arXiv preprint arXiv:2409.18025*, 2024.

- Pratyush Maini, Zhili Feng, Avi Schwarzschild, Zachary Chase Lipton, and J Zico Kolter. Tofu: A task of fictitious unlearning for llms. In *Conference on Language Modeling*, 2024.
- Neil G Marchant, Benjamin IP Rubinstein, and Scott Alfeld. Hard to forget: Poisoning attacks on certified machine unlearning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 7691–7700, 2022.
- Daniel Marczak, Bartłomiej Twardowski, Tomasz Trzciński, and Sebastian Cygert. Magmax: Leveraging model merging for seamless continual learning. In *European Conference on Computer Vision*, pp. 379–395. Springer, 2024.
- Michael S Matena and Colin A Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.
- Yash More, Prakhar Ganesh, and Golnoosh Farnadi. Towards more realistic extraction attacks: An adversarial perspective. *arXiv preprint arXiv:2407.02596*, 2024.
- Guillermo Ortiz-Jimenez, Alessandro Favero, and Pascal Frossard. Task arithmetic in the tangent space: Improved editing of pre-trained models. *Advances in Neural Information Processing Systems*, 36, 2024.
- Formerly Data Protection. General data protection regulation (gdpr). *Intersoft Consulting*, Accessed in October, 24(1), 2018.
- Hongyu Qiu, Yongwei Wang, Yonghui Xu, Lizhen Cui, and Zhiqi Shen. Fedcio: Efficient exact federated unlearning with clustering, isolation, and one-shot aggregation. In *2023 IEEE International Conference on Big Data (BigData)*, pp. 5559–5568. IEEE, 2023.
- George Stoica, Daniel Bolya, Jakob Bjorner, Pratik Ramesh, Taylor Hearn, and Judy Hoffman. Zipit! merging models from different tasks without training. *arXiv preprint arXiv:2305.03053*, 2023.
- Ellen Su, Anu Vellore, Amy Chang, Raffaele Mura, Blaine Nelson, Paul Kassianik, and Amin Karbasi. Extracting memorized training data via decomposition. *arXiv preprint arXiv:2409.12367*, 2024.
- Ayush K Tarun, Vikram S Chundawat, Murari Mandal, and Mohan Kankanhalli. Fast yet effective machine unlearning. *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jimenez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. *arXiv preprint arXiv:2405.07813*, 2024.
- Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pp. 23965–23998. PMLR, 2022.
- Leijie Wu, Song Guo, Junxiao Wang, Zicong Hong, Jie Zhang, and Yaohong Ding. Federated unlearning: Guarantee the right of clients to forget. *IEEE Network*, 36(5):129–135, 2022.
- Xiaoyu Xia, Ziqi Wang, Ruoxi Sun, Bowen Liu, Ibrahim Khalil, and Minhui Xue. Edge unlearning is not “on edge”: an adaptive exact unlearning system on resource-constrained devices. *arXiv preprint arXiv:2410.10128*, 2024.
- Zuobin Xiong, Wei Li, Yingshu Li, and Zhipeng Cai. Exact-fun: An exact and efficient federated unlearning approach. In *2023 IEEE International Conference on Data Mining (ICDM)*, pp. 1439–1444. IEEE, 2023.
- Zhengqi Xu, Ke Yuan, Huiqiong Wang, Yong Wang, Mingli Song, and Jie Song. Training-free pretrained model merging. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5915–5925, 2024.

- Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36, 2024a.
- Prateek Yadav, Tu Vu, Jonathan Lai, Alexandra Chronopoulou, Manaal Faruqui, Mohit Bansal, and Tsendsuren Munkhdalai. What matters for model merging at scale? *arXiv preprint arXiv:2410.03617*, 2024b.
- Haonan Yan, Xiaoguang Li, Ziyao Guo, Hui Li, Fenghua Li, and Xiaodong Lin. Arcane: An efficient architecture for exact machine unlearning. In *IJCAI*, volume 6, pp. 19, 2022.
- Peng Ye, Chenyu Huang, Mingzhu Shen, Tao Chen, Yongqi Huang, Yuning Zhang, and Wanli Ouyang. Merging vision transformers from different tasks and domains. *arXiv preprint arXiv:2312.16240*, 2023.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.
- Mingyang Zhang, Jing Liu, Ganggui Ding, Xinyi Yu, Linlin Ou, and Bohan Zhuang. Channel merging: Preserving specialization for merged experts. *arXiv preprint arXiv:2412.15283*, 2024.

Appendices

A Algorithm Pseudocode

Algorithm details. One key detail of SIFT-Masks is that finetuning must be fully deterministic. In order to guarantee that a model is properly unlearned, we must obtain the original weights that were used during merging and subtract them from the multi-task vector.

Algorithm 1: PyTorch-like pseudocode for SIFT-Masks

```

1 Require:  $T$  (task indices),  $\{c_t\}_{t \in T}$  (tasks),  $M_0$  (pretrained model),  $E_{\text{tune}}$  (finetuning
  steps),  $\eta$  (learning rate)
2 Function SIFT( $t, v$ ):
3    $\tau_t \leftarrow \vec{0}$ 
4    $\text{opt} \leftarrow \text{torch.optim.Adam}(\tau_t, \eta)$ 
5   for  $i = 1..E_{\text{tune}}$  do
6      $M_t = M_0 + \tau_t$ 
7      $x, y \leftarrow \text{sample batch of data from } c_t$ 
8      $\text{loss} \leftarrow \text{torch.nn.cross\_entropy\_loss}(M_t(x), y)$ 
9      $\text{loss.backward}()$ 
10     $m_t \leftarrow \mathbb{1}\{\tau_t \odot v > 0\}$ 
11     $\tau_t \leftarrow \tau_t \odot m_t$ 
12     $\text{opt.step}()$ 
13  return  $\tau_t, m_t$ 
14 Function Merge( $\{\tau_t\}_{t \in T}$ ):
15    $\bar{\tau} \leftarrow \sum_{t \in T} \tau_t$ 
16  return  $\bar{\tau}$ 
17 Function Unmerge( $\bar{\tau}, t$ ):
18    $\bar{\tau} \leftarrow \bar{\tau} - \tau_t$ 
19    $T \leftarrow T \setminus \{t\}$ 
20  return  $\bar{\tau}$ 
21 Function Localize( $\bar{\tau}, t$ ):
22  return  $(\bar{\tau} \odot m_t) / |T|$ 
23 Function SIFT-masks( $T$ ):
24    $v \leftarrow \mathbb{1}\{\text{torch.rand\_like}(M_0) > 0.5\}$ 
25   for  $t = 1..T$  do
26      $\tau_t, m_t = \text{SIFT}(t, v)$ 
27    $\bar{\tau} \leftarrow \text{Merge}(\{\tau_t\}_{t \in T})$ 
28  return  $\bar{\tau}$ 

```

B Additional Details for Experiments

Reproducibility Statement In this section, we include details on our experimental setup. Each method uses a fixed set of hyperparameters in all the experiments it appears in; we only use different hyperparameters depending on the dataset and model. We provide all relevant information on algorithm details and hyperparameter configurations in order to reproduce Algorithm 1. Upon acceptance of this work, we will publically share our source code and data preprocessing setup.

Dataset overview. In Table 3, we provide more complete details on the datasets and models used in our experiments. GPT2-XL has 1.5B parameters, Llama3.2-1B-Instruct has 1.2B parameters, and FLAN-T5-Large has 700M parameters. We run full finetuning on all hidden layers of the model i.e. freeze the embedding / language modeling head.

Data access and preprocessing. Next, we briefly explain the setup for each dataset.

Dataset	Model	Labels (per task)			Examples (per task)		
		Max	Total		Min	Max	Total
TOFU	GPT2-XL	200	-	50256	20	20	4000
Sent140	Llama3.2-1B-Instruct	500	2	2	52	440	40504
Reddit	FLAN-T5-Large	500	12	20	4	718	67484
StackOverflow	FLAN-T5-Large	500	10	10	124	632	97037

Table 3: More details on the datasets in our experiments. Each task besides TOFU has a minimum of 2 labels (TOFU is a next-token prediction task).

TOFU. The TOFU dataset is readily accessible from Huggingface: <https://github.com/locuslab/tofu>. We do not run any preprocessing or filtering on this dataset.

Sent140. The download script for Sentiment140 can be found on Huggingface at <https://huggingface.co/datasets/stanfordnlp/sentiment140>. We do not run any preprocessing or filtering on this dataset.

Reddit. We use a dump of Reddit comments from December 2017. A public version of this data can be found as in the LEAF benchmark dataset at <https://github.com/TalwalkarLab/leaf/tree/master/data/reddit>. For our experiments, we obtained the original dump from academictorrents.net and will share our preprocessed version once the paper is made public.

1. For our preprocessing, we identified 20 popular subreddits, excluding a few subreddits where there was significant overlap of content (e.g. “r/politics” and “r/news”).
2. Our final list of subreddits is as follows: ['politics', 'nfl', 'nba', 'soccer', 'Bitcoin', 'StarWars', 'DestinyTheGame', 'movies', 'leagueoflegends', 'fantasyfootball', 'hockey', 'teenagers', 'RocketLeagueExchange', 'MMA', 'FireEmblemHeroes', 'NintendoSwitch', 'Overwatch', 'relationships', 'pathofexile', 'anime'].
3. We then sampled the top 500 users with the most posts across all these subreddits while excluding accounts containing keyphrases indicating that they are bots.
4. The list of bot keywords we used was: ["auto", "mod", "bot", "ImagesOfNetWork", "MTGCardFetcher", "tippr", "DreamProcessor", "Mentioned_Videos", "keepdankmemesdank", "User_Simulator", "AreYouDeaf", "ThisCatMightCheerYou", "TotesMessenger", "transcribersofreddit", "xvicsagex", "notifier", "Roboragi", "robot"].

StackOverflow. The data for StackOverflow can be found at Huggingface at <https://huggingface.co/datasets/mikex86/stackoverflow-posts>. We first identified the following top 10 tags: ['javascript', 'python', 'java', 'c#', 'php', 'android', 'html', 'jquery', 'c++', 'css']. We then subsampled the data to only include comments which were labeled with only one of these 10 tags. From this subset, we then sampled the 500 users with the most comments.

Prompting and finetuning setup. For each dataset, we include a prompt which allows the model to achieve non-trivial zeroshot accuracy. Each model is then trained using a causal language modeling objective. That is, instead of randomly initializing and finetuning a classifier head, we keep the original language modeling head and train the model to output the text corresponding to a label i.e. “positive” or “negative” for Sent140, or one of the subreddit or tags listed above for Reddit or StackOverflow respectively.

TOFU. Each example in TOFU is a question-answer pair (q, a) which we transform using a simple prompt: “Question: {q}\nAnswer: {a}”. We then finetune the model to predict the tokens corresponding to the answer sequence a.

Sent140. Each example is a comment with a binary label indicating whether the comment is positive or negative sentiment. We format this using the following Llama3 prompt:

```

<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are an AI assistant designed for sentiment analysis.
<|eot_id|><|start_header_id|>user<|end_header_id|>
What is the sentiment of this comment? Respond with a single word.
Do not start with 'The'. Choices:
- Negative
- Positive
Comment: {comment}
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
{label}

```

Reddit and StackOverflow. Like for Sent140, we provide a prompt that includes the list of 20 subreddits or 10 tags we filtered during preprocessing.

```

Context:{comment}
Question:Choose the most relevant topic.
OPTIONS:
{option_list}
Answer: {label}'

```

Hyperparameters. Next, we list out the hyperparameters that we tuned for our method and other baselines in the experiments. Generally, we used the same values for all shared hyperparameters across methods (i.e. learning rate, batch size, finetuning steps).

Finetuning. As stated in Section 4.2, we used 20 finetuning steps for each individual task and up to 800 finetuning steps for Central FT. For TOFU, we finetune with a batch size of 20 (each task contains exactly 20 examples). For the other three datasets, we finetune with a batch size of 128; if a task has less than 128 examples, we use full-batch gradient descent. For TOFU, we use a learning rate of $1e-4$. For Reddit and StackOverflow, we use a learning rate of $1e-5$, and for Sent140, we use a learning rate of $1e-7$.

Merging. In our work we use a simple unweighted average of task vectors, as shown in the “Localization” step of Algorithm 1. Although prior works in model merging propose rescaling the average or using a weighted average of the task vectors, we found that these additional techniques were costly to tune and did not result in significant improvements.

Localization. For TALL-masks, we tuned the threshold hyperparameter λ based on the density of the mask rather than the threshold value, in order to account for varying magnitudes of merged weights across datasets. We performed a grid search over density values $[0.1, 0.3, 0.5, 0.7, 0.9]$. For a fair comparison to our method, we consider rescaling the masked model by a hyperparameter α which is also tuned via grid search over values $[0.8, 1, 1.2, 1.4, 1.6]$.

C Additional experiments

Model selection and data formatting matters. On both Reddit and Sent140, we find that model size is not necessarily the most important factor for good merging. In particular, it is important to consider (1) model architecture and (2) modeling format. **With the proper modeling format, smaller models can outperform larger architectures from other model families.** Generally, we see that using (1) an instruction-tuned model, (2) a language modeling rather than classification objective and (3) a prompt that achieves good zero-shot performance can greatly improve the performance of merging. “CLS Head” means we finetune a classification head, “ZS Init” means we initialize the classification heads using an embedding of the label text, “LM Head” means we keep the original language modeling head and finetune the model to output the label text, and “Prompt” means we design a prompt that encourages the model to output the label texts.

Sent140					
Model	Params	Format	Zeroshot	Finetune	Merge
GPT2-XL	1.61B	CLS Head	50.0	85.7	70.8
Llama 3.2-1B	1.24B	CLS Head	50.0	69.4	55.3
		CLS Head + ZS Init	50.0	67.4	53.0
		LM Head + Prompt	68.5	85.5	77.1

Table 4: Merging 100 clients on Sent140. GPT2-XL and Llama 3.2 can achieve similar local accuracy after finetuning, but Llama 3.2 has much better accuracy after merging.

Reddit					
Model	Params	Format	Zeroshot	Finetune	Merge
Llama 3.2-1B	1.24B	LM Head + Prompt	10.0	75.7	24.9
FLAN-T5-Large	783M	CLS Head	3.0	61.3	19.3
		CLS Head + ZS Init	7.1	61.8	16.5
		CLS Head + ZS Init + Prompt	6.7	70.5	21.7
		LM Head + Prompt	29.7	76.5	43.2

Table 5: Merging 100 clients on Reddit. While Llama 3.2 does well on Sent140, it does poorly on Reddit, indicating that testing multiple model architectures can be helpful. We do not test FLAN-T5 on Sent140 because Sent140 is included in the FLAN dataset. Unlike the Reddit setting presenting in the main paper, here we ran experiments on a setting where each client’s data is subsampled to only hold 50 examples from each of its 2 most common labels.

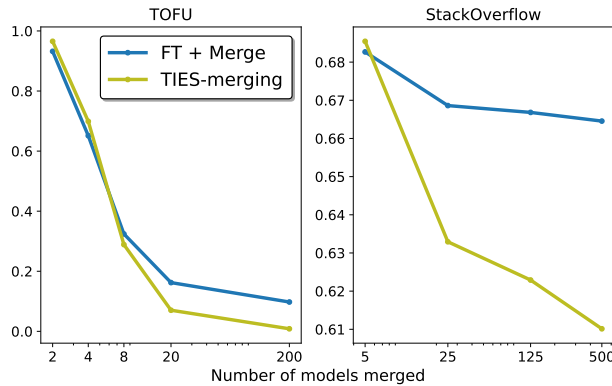


Figure 11: We compare TIES-merging to FT + Merge while varying the number of models merged. Scale can disproportionately harm TIES-merging compared to regular FT + Merge.

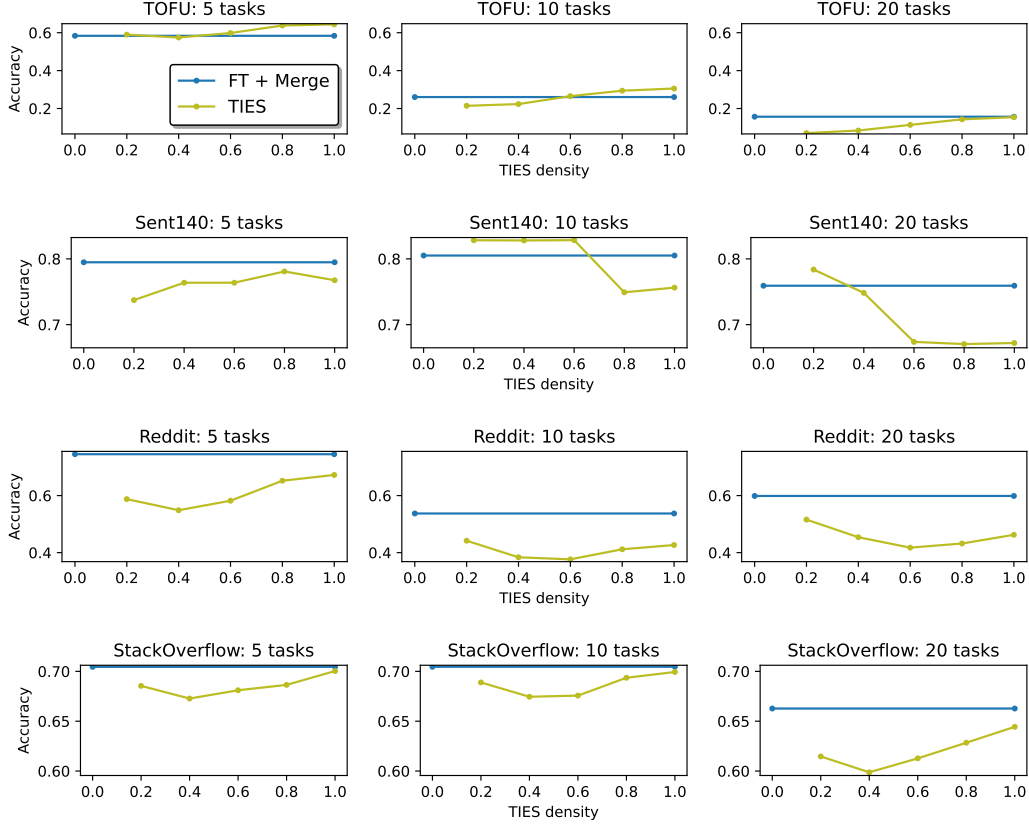


Figure 12: We compare TIES-merging to FT + Merge while varying the density of TIES-merging and the number of models merged. First, sparsity (trimming) tends to be more beneficial when merging a larger number of models. However, in most settings, sparsity harms the accuracy of individual models and outweighs the benefits of reducing sign conflicts.

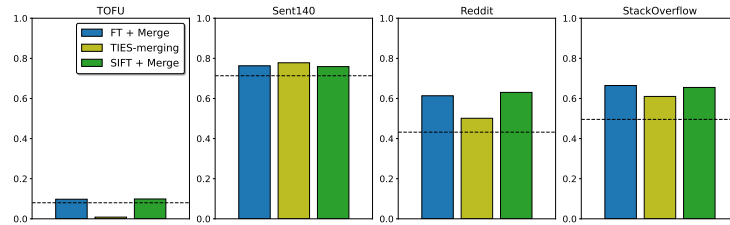


Figure 13: We compare TIES-merging to FT + Merge and SIFT + Merge when merging 500 models. While both TIES-merging and SIFT + Merge attempt to reduce sign conflicts, this does not significantly affect merging quality.